

Distributed Systems Project Report: Agentic Kubernetes Management

Andrej Čop

December 11, 2025

1 Introduction

This project explores the intersection of Large Language Models LLM based AI agents and Distributed Systems orchestration. It serves as a foundation for a Master's Thesis project focused on automating the lifecycles of ML models inference services titled: *AI Agent-Driven Control of ML Models as a Service*.

The domain of AI agents for cloud infrastructure is quickly evolving. Recent works (*Cloud Infrastructure Management Agents* [2] and *KubeIntellect* [1]) have demonstrated the potential of LLMs to automate DevOps tasks. Existing solutions often lack the specific context required for managing Machine Learning models as services, such as GPU allocation, model versioning, and inference latency based scaling.

This project implements an agent capable of managing a Kubernetes cluster through natural language interaction. This is what we defined as level 1 complexity of AI agents, while levels 2 and 3 would involve modifications of services based on monitoring metrics and a fully autonomous, self-healing system with minimal human intervention, respectively.

This project abstracts the complexity of the 'kubectl' command, allowing users to interact with infrastructure using natural language. The system democratizes access to distributed systems by enabling data scientists to deploy models without expertise.

2 Architecture and Implementation

The system is built as a local solution using Python and the CrewAI framework and can work on a single Linux machine or generalise to a full Kubernetes cluster with an external LLM API.

2.1 Components

- **Kubernetes:** System is a local Kubernetes cluster running via **Kind** (Kubernetes in Docker). A custom 'kind-config.yaml' maps container ports to localhost so we can test the example service outside the cluster.
- **Agent Framework:** A single CrewAI agent acts as a *Kubernetes assistant*. It is designed with extensibility for the master's thesis in mind, allowing for a multi-agent architecture where specialised agents (configuration writer, documentation reader, monitoring agent) collaborate.
- **Tooling:** A custom 'KubectlTool' was implemented in Python with the CrewAI library. It executes shell commands against the cluster and captures output. It feeds the execution results back to the LLM. This feedback loop allows the agent to correct if a command fails and summarise findings back to the user.
- **LLM:** The architecture supports switching between models. Experiments were conducted using **qwen3:14B** running locally via **Ollama** and **claude-3-sonnet** via **OpenRouter**. While OpenRouter offers free models, it has strict rate limits. The local **qwen3:14B** model showed better performance.
- **Simulated ML Workload:** To validate the agent's ability to handle ML inference scenarios, an 'echo_service' was developed. It is a simple echo service using FastAPI, Uvicorn server and containerised with Docker and performs computational load (matrix multiplication) to simulate the latency profile of real ML inference services. The service's pod is constrained to a single CPU core. It can be easily extended with an ML library such as Pytorch and with a real pre-trained ML model. Our choice to use simple matrix multiplication was to minimize the docker image size for testing purposes.

2.2 Agent Memory

The application implements a memory for agents. It retains a configurable number of recent prompts and tool outputs. This helps the agent to remember its actions and keep the interaction natural.

3 Usage

The project is designed to be run on a single Linux machine with Docker, Kubectl, Kind, and Python installed.

3.1 Setup and Execution

To set up the environment, users must first install dependencies (using ‘uv sync’ or pip) and configure the LLM provider by creating a ‘.env’ file from the provided example. The local Kubernetes cluster is then initialised with a custom configuration.

```
kind create cluster --name agent-cluster --config kind-config.yaml
```

Docker image for the simulated ML workload should be built and copied in to the cluster:

```
docker build -t echo-service:latest ./echo_service
kind load docker-image echo-service:latest --name agent-cluster
```

Once the infrastructure is ready, the agent is started via ‘python main.py’. The user interacts with the system through a terminal-based chat interface. Supported interactions range from informational queries to complex deployment tasks. Example prompts include:

- “Who are you?”
- “Can you check the status of the cluster and all the pods and report back?”
- “Deploy an example hello world nginx service”
- “Deploy a deployment using config file located in echo_service/deployment.yaml”

We suggest running `watch -n 1 kubectl get pods` command in the background to monitor the cluster while running the above prompts.

Executing the last prompt will deploy an example service that simulates an inference workload (assuming the docker image was built and is inside the cluster). To validate the deployment, a load testing script is provided. Running ‘python load.test.py’ generates concurrent traffic to the deployed service, measuring latency to simulate real-world inference workloads. Ask the agent to increase the replicas and see the change in results.

Results Running the load testing script on the service with 20 requests, sending 5 concurrently, the average latency was 8.3s. After telling the agent: *Can you increase the replicas of the service to 3?* The agent performed the task successfully, and the average latency with the same benchmark parameters was 3.3s.

4 Challenges

The main challenge was the so-called “hallucinations” where the agent claims actions were taken without execution. To fix this, the system prompt has to be engineered so that the agent’s behaviour is strict. The agent has to verify all actions with ‘kubectl’ before reporting success (*backstory* argument of *Agent* object in *main.py*)

The current implementation does not have guardrails which prevent the agent from executing destructive commands. These include syntax validation to block destructive commands ‘delete’, and human in the loop confirmation for high impact actions. While we have not encountered such destructive actions during testing, this has to be kept in mind for the final thesis project solution.

References

- [1] Mohsen Seyedkazemi Ardebili and Andrea Bartolini. KubeIntellect: A modular LLM-orchestrated agent framework for end-to-end Kubernetes management. *arXiv preprint arXiv:2509.02449*, December 2024.
- [2] Zhenning Yang, Archit Bhatnagar, Yiming Qiu, Tongyuan Miao, Patrick Tser Jern Kon, Yunming Xiao, Yibo Huang, Martin Casado, and Ang Chen. Cloud infrastructure management in the age of ai agents. *SIGOPS Oper. Syst. Rev.*, 59(1):1–8, August 2025.